22BCON1068

Assignment – 4

Sheet – 08

13/04/24

DATE: / /

PAGE NO.: 47

- Sec-A

1. What are the 4 Properties of transaction database ?

Ans The 4 properties of a transaction are often referred to as ACID properties.

(i) Atomicity : The entire transaction takes Place at once.

(ii) Consistency : Transactin maintain data integrity.

(iii) Isolation : Transaction Operate independently.

(iv) Durability : committed transactions persist even after system failure.

2. What is cascading rollback ?

Ans cascading rollback is a domino effect in databases. When one transcation fails and needs to be undon(roll back), other transaction that depended on it also need to be rolled back, creating a chain reaction.

3. What is timestamp ?

Ans A timestamp is a uniaue identifier assigned to each transaction in a database, helping to track the order of operations and manage concurrency control.

4. Discuss about view Serializability ?

Ans procedure to check if the given schedule is consistant or not. view seriallizability ensures that concurrent transactions produce the same result as if they were executed seauentially, maintaining data integrity and consistency.

5. What is recoverable and irrecoverable schedule.

Ans A recoverable schedule is one in which transactions can be rolled back if needed, Preserving the integrity of the database in the event of a failure.

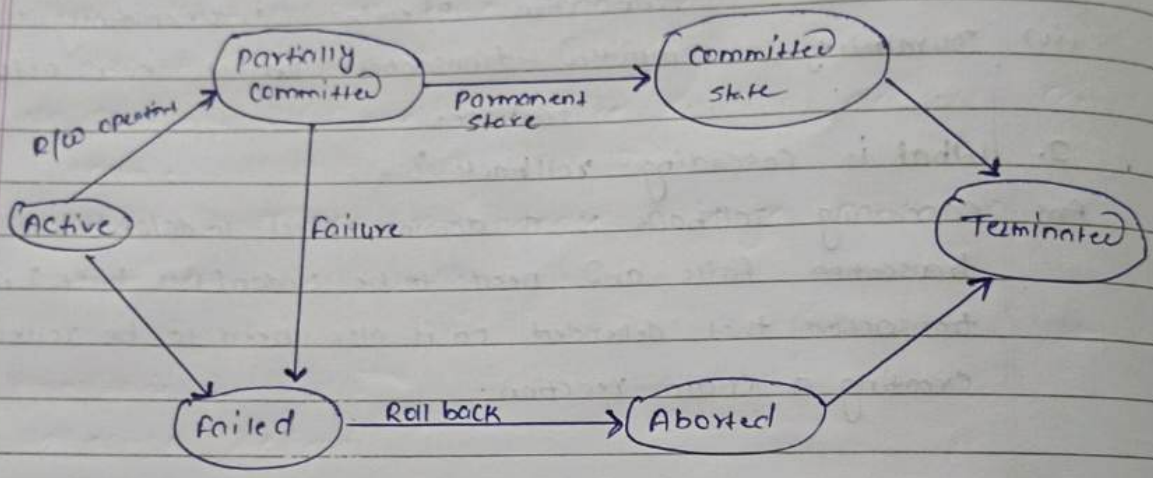
May 8, 2024, 17:56

An irrecoverable schedule is one in which transactions can't be rolled back, potentially leading to data inconsistencies if a failure occurs.

- Sec-B (3×7=21)

1. Explain different states of transactions with diagram.

Ans



(Transaction state)

(i) Active: in this state, the transaction is actively performing operations on the database.

(ii) partially committed: The transaction has executed all its operations successfully and is ready to commit.

(iii) Committed: The transaction has completed successfully, and all its changes made by are now permanently saved in the database.

(iv) Failed: if any error occurs during execution, the transaction fails.

(v) Aborted: After a failure, the system rolls back all the changes made by the transaction.

May 8, 2024, 17:56

(vi) Terminated : This is the final state, reached after the transaction is either commited or aborted.

2. Explain in detail about timestamp based concurrency control techniques.

Ans Timestamp-based concurrency control (T-BCC) is an optimistic approach to managing concurrent transactions in databases. Unlike locking mechanisms, it allows transaction to proceed without acquiring locks and validates them later based on timestamps. This approach offers advantages like improved concurrency and avoiding deadlocks.

- Concept :

(i) Timestamps : Each transaction is assigned a unique timestamp when it starts. This timestamp act as a version identifier and determines the transaction's order of execution.

(ii) Read and Write Operations :

- Read : When a transaction reads data, it compares its timestamps with the write timestamp of that data item.
- Valid Read : if the transaction's timestamp is greater than the write timestamp, the read is considered valid.

- Write validation : The actual data update is deferred until the transaction attempts to commit.

- Write : When a transaction attempts to commit :

→ valid write : if no other ~~validated~~ commited transaction has a conflicting write timestamp, the write is valid.

- Write Invalidation: if a conflicting write with an earlier timestamp is found, the current transaction is invalidated (rollbacked).

- Benefits:
- High Concurrency: Transaction can proceed without acquiring locks, leading to better concurrency compared to locking method.
- No Deadlocks: Since there are no locks, deadlocks cannot occur.
- Scalability: T-BCC can handle a large number of transaction efficiently.

- Challenges:
→ Validation Overhead
→ Write Invalidation
→ Timestamp Synchronization

- Types of Timestamp-Based Concurrency control protocols: There are two main protocol based on how they handle validation:

(i) Timestamp ordering Protocol:
→ Reads are validated only if the transaction's timestamp is greater than all read and write timestamps of the data item.
→ Offers stricter consistency but can lead to more write Invalidation.

(ii) Read Committed Protocol:
→ Reads are validated only if the transaction's timestamp is greater than all write timestamps of the data item.
→ Less strict consistency but reduces write Invalidation.

DATE : / /
PAGE NO. : 51

3. Consider the following schedule S.

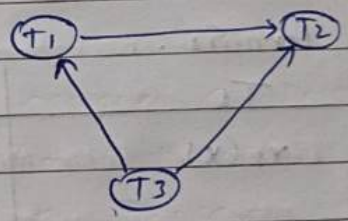
S: $R_1(x)$; $R_2(z)$; $R_1(z)$; $R_3(x)$; $R_3(y)$; $W_1(x)$; $W_3(y)$; $R_2(y)$; $W_2(z)$;

In the precedence graph of S, calculate number of edge (both incoming and outgoing) of nodes T_1 and T_2.

Ans

$R_1(x) \rightarrow W_1(x)$ (T_1)
$R_1(x) \rightarrow R_3(x)$ (T_1)
$R_2(z) \rightarrow W_2(z)$ (T_2)
$R_2(z) \rightarrow R_2(y)$ (T_2)
$R_1(z) \rightarrow R_1(z)$ (T_1)
$R_3(x) \rightarrow R_3(y)$ (T_1)
$R_3(y) \rightarrow W_3(y)$ (T_1)
$W_1(x) \rightarrow W_1(x)$ (T_1)
$W_3(y) \rightarrow W_3(y)$ (T_1)

Precedence graph:



for T_1 :-

Incoming edge : 1 ~~(From $R_1(x)$, $R_3(x)$ and $R_3(y)$)~~

outgoing edge : 1 ~~(From $W_1(x)$, $R_1(x)$, $R_3(x)$, $W_3(y)$)~~

for T_2 :-

Incoming edges: 2 (From $R_2(z)$, $R_2(z)$)

outgoing edges: 0 ~~(To $W_2(z)$ and $R_2(y)$)~~

May 8, 2024, 17:56

DATE : / /
PAGE NO. : 52

• Sec- C (3X11=33)

1. Consider the transaction T1, T2, and T3 and the schedules S1 and S2 given below.

T1 : r1(x); r1(z); w1(x); w1(z)

T2 : r2(y); r2(z); w2(z)

T3 : r3(y); r3(x); w3(y)

S1 : r1(x); r3(y); r3(x); r2(y); r2(z); w3(y); w2(z); r1(z); w1(x); w1(z)

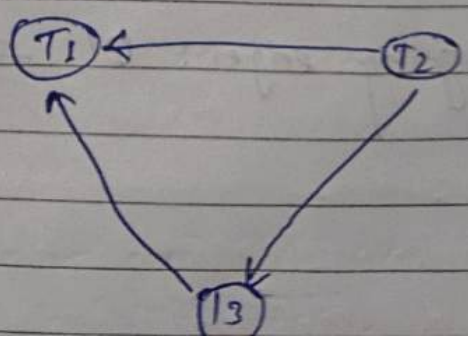
S2 : r1(x); r3(y); r2(y); r3(x); r1(z); r2(z); w3(y); w2(x); w2(z); w1(z)

Which schedule is/are conflict serializable? Draw Precedence graph and Justify your answer.

Ans For S1 :

T1	T2	T3
r1(x)		
		r3(y)
		r3(x)
	r2(y)	
	r2(z)	
		w3(y)
	w2(z)	
r1(z)		
w1(x)		
w1(z)		

Precedence graph :



Q. ~~Explain various types of~~

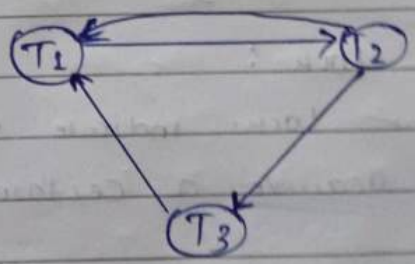
There is no cycle or loop present in Precedence graph hence S1 is a Conflict serializable.

order : $T_2 \rightarrow T_3 \rightarrow T_1$

- For S2 :

T_1	T_2	T_3
r(x)		
		r(y)
	r(y)	
		r(z)
r(z)		
	r(z)	
		w(y)
w(x)		
	w(z)	
w(z)		

precedence Graph :



There is a loop in S2 schedule so it is not Conflict serializable.

Q3. Explain various types of locks in Detail.

Ans Locks are a fundamental mechanism used in database management system to control access to shared resources, ensuring data integrity and consistency in multi-user environments. Here's an explanation of some common types of locks:

(i) Shared lock (S-lock):
→ Also known as read locks.
→ Shared locks allow multiple transactions to read a resource simultaneously.
→ Shared locks are compatible with other shared locks but incompatible with exclusive locks (write locks).

(ii) Exclusive lock (X-lock):
→ Also known as write locks.
→ Exclusive lock grant exclusive access to a resources preventing any other transaction from reading or writing to it concurrently.

(iii) Intent lock:
→ Intent locks indicate the intention of a transaction to acquire a certain type of lock on a resource.

→ These locks are used to prevent deadlocks by indicating the future locking intentions of a transaction.

→ it can be two types: Intent shared (IS) locks and Intent exclusive (IX) locks.

(iv) Update Lock (U-Lock):

→ Update lock are a combination of shared and exclusive locks.

→ They are acquired when a transaction intends to update a resource but does not want other transaction to acquire exclusive locks on it until the update is completed.

(v). Schema Lock: Schema lock are used to control concurrent access to database schema objects such as table, views, or stored procedures.

→ Schema locks are often acquired implicitly by transactions when accessing schema objects.

(vi) Shared intent exclusive (SIX) Lock :-

→ SIX locks are used to indicate the intention of a transaction to perform an update operation after reading a resource.

→ Six locks are compatible with shared locks but incompatible with exclusive locks.

- There are four type of lock protocol available :
  (i) Simplistic Lock protocol
  (ii) Pre-Claiming Lock protocol
  (iii) Two-phase locking (2PL)
  (iv) Strict Two Phase locking (Strict-2PL)

DATE: / /
PAGE NO.: 56

Q2. (a) Identify the name of problem in given schedule and Justify your answer with the help of suitable example.

T_1	T_2
R(x)	
W(x)	
R(y)	
	W(x)
	R(y)
	Commit
Commit	

→ The problem in this schedule is lost update problem. Specifically, Transaction (T_2) writes data item X after transaction (T_1) has already written to X but before T_1 commits. This result in the update made by T_1 being lost as T_2's update overwrites it before T_1's changes the committed.

Ex: So the lost update occurs when T_2's write operation on X (let X=4) overwrites the previous value written by T_1 (let X=2), leading to the loss of T_1's update.

(b) Explain the following Time stamping protocol with the help of suitable examples.

(i) Ti Request for Read(B)

(ii) Ti Request for Write(B).

May 8, 2024, 17:57

(i) Ti Request for Read (Q):

The system compares the timestamp of Ti (TS(Ti)) with the write timestamp (W-TS(Q)) of data item Q. if TS(Ti) > W_TS(Q), it means Ti started after the last write operation on Q. This allows read to proceed, and the read timestamp (R_TS(Q)) of Q is updated to maximum of the current R_TS(Q) and TS(Ti). if TS(Ti) <= W_TS(Q), it means Ti started before or at the same time as the last write on Q. This could lead to inconsistencies if Ti reads an uncommitted value.

Ex: Let Ti request to read data item Q. if TS(Ti) is 10 and the last update timestamp of Q, W_TS(Q) is 8, Ti can proceed with the read operation. However, if W_TS(Q) is 12, Ti must wait for the update to be committed before reading Q.

(ii) Ti Request for write (Q):

The system compares the timestamp of Ti (TS(Ti)) with both R_TS(Q) and W_TS(Q).
if TS(Ti) > R_TS(Q) and TS(Ti) > W_TS(Q), it means no other transaction has read or written Q after Ti started. The write proceeds and W_TS(Q) is updated to TS(Ti).
if TS(Ti) <= W_TS(Q), another transaction has already written Q. This can lead to inconsistency.

May 8, 2024, 17:57

Ex: Let Ti requests to write data item Q. If TS(Ti) is 15 and last read of timestamp of Q, R_TS(Q) is 12, Ti can proceed with write operation. After writting Q, Ti updates the timestamp of Q to 15 (to W_TS(Q)=15).